LLM API 速查表 · LLM API Cheatsheet

OpenAI · Anthropic · Google · SDK 三家速查 · 2026.05

OpenAI Chat Completions — OpenAI

安装 / Setup

```
pip install openai
from openai import OpenAI
client = OpenAI(api_key=...)
```

Chat Completions(经典)

```
r = client.chat.completions.create(
  model="gpt-5",
  messages=[
    {"role":"system","content":"..."},
    {"role":"user","content":"hi"}],
  tools=[...], temperature=0.7,
  stream=True)
print(r.choices[0].message.content)
```

Responses API(新)

```
r = client.responses.create(
  model="gpt-5",
  input="...",
  instructions="...",
  tools=[{"type":"web_search"}])
r.output_text
```

JSON Schema 结构化输出

```
response_format={
  "type":"json_schema",
  "json_schema":{"name":"X",
    "schema":{...},"strict":True}}
```

关键点

system 角色在 Responses API 改名
`developer` ; `parallel_tool_calls` 默认 True

Anthropic Messages — Anthropic

安装 / Setup

```
pip install anthropic
from anthropic import Anthropic
client = Anthropic(api_key=...)
```

Messages 主接口

```
r = client.messages.create(
  model="claude-opus-4-7",
  system="You are ...",
  messages=[
    {"role":"user","content":"hi"}],
  tools=[...],
  max_tokens=4096,
  temperature=1)
r.content[0].text
```

Content Blocks(关键)

`text`	普通文字
`image`	base64 / URL 图片
`tool_use`	模型发起调用
`tool_result`	回填工具输出
`thinking`	扩展思考块

Prompt Caching

```
{"type":"text","text":"...",
 "cache_control":{"type":"ephemeral"}}
# 5min / 1h, 写入 +25%, 读 -90%
```

Gemini API — Google

安装 / Setup(新 SDK)

```
pip install google-genai
from google import genai
from google.genai import types
client = genai.Client(api_key=...)
```

generate_content

```
r = client.models.generate_content(
  model="gemini-2.5-pro",
  contents=["hi"],
  config=types.GenerateContentConfig(
    system_instruction="...",
    tools=[...],
    temperature=0.7,
    response_mime_type=
      "application/json",
    response_schema=Pydantic))
r.text
```

Context Caching

```
cache = client.caches.create(
  model="gemini-2.5-pro",
  config=types.CreateCachedContentConfig(
    contents=[...],ttl="3600s"))
# 用 cache.name 复用,长上下文省钱
```

关键点

老 `google-generativeai` 已弃用; `contents` 列表元素用 `Part` / 字符串均可

流式输出 — Streaming (SSE)

协议

三家都用 **SSE**(Server-Sent Events),HTTP 长连接,逐 chunk 推送 `data:` 帧;SDK 把 SSE 包装成迭代器

OpenAI(增量 delta)

```
stream = client.chat.completions
  .create(..., stream=True)
for chunk in stream:
  d = chunk.choices[0].delta.content
  if d: print(d, end="")
```

Anthropic(累积 + helpers)

```
with client.messages.stream(...) as s:
  for text in s.text_stream:
    print(text, end="")
  msg = s.get_final_message()
```

Gemini

```
for chunk in client.models
  .generate_content_stream(...):
  print(chunk.text, end="")
```

累积 vs 增量

`OpenAI delta`	需自行拼接 content
`Anthropic event`	message_start/delta/stop
`Gemini chunk`	每片是完整 Response 切片

工具调用 — Tool Use / Function Calling

Schema(三家相似)

基于 **JSON Schema**: `name` / `description` / `parameters` 或 `input_schema`

OpenAI

`tools=[{"type":"function",...}]`	函数定义
`strict: true`	强制 schema 合法
`parallel_tool_calls`	默认 True
`tool_choice="auto"`	auto/required/none/{...}

Anthropic

`input_schema`	JSON Schema
`disable_parallel_tool_use`	关并行(默认开)
`tool_choice={"type":"auto"}`	auto/any/tool/none

Gemini

`FunctionDeclaration`	types.Tool 包装
`tool_config`	AUTO/ANY/NONE

多轮回填

把 **assistant 的 tool_use** + **user 的 tool_result** 一起 append 回 messages,再 call;直到无 tool_use 即结束

多模态 / 视觉 — Vision / Multimodal

OpenAI(image_url)

`{"type":"image_url",...}`	URL 或 data:base64
`image_url.detail`	low / high / auto

Anthropic(source)

`{"type":"image","source":{...}}`	base64 或 url 类型
`media_type`	image/png/jpeg/webp/gif

Gemini(Part)

`Part.from_bytes(data,mime)`	直接传 bytes
`Part.from_uri(uri,mime)`	GCS / Files API URI

URL 图片

三家都接受 **https://** URL,服务器侧抓取(私网/防盗链需走 base64)

PDF 文档

`Anthropic`	原生 document 块,逐页视觉化
`OpenAI`	Files API 上传 → 在 messages 引用
`Gemini`	直接传 PDF Part(**2GB/文件**)

音视频

Gemini 原生支持音视频 Part;OpenAI 需 Whisper / GPT-4o audio;Anthropic 暂无音频

错误处理 — Errors & Retry

常见状态码

`400`	参数/schema 不合法,不要重试
`401 / 403`	key / 权限 / 区域问题
`404`	model 名拼错或无访问
`413 / 422`	payload 过大 / 不可处理
`429`	rate limit,看 **Retry-After** header
`5xx / 529`	服务过载,指数退避 + jitter

SDK 内置重试

`OpenAI(max_retries=2)`	默认 2 次,指数退避
`Anthropic(max_retries=...)`	同上,只重试 408/429/5xx
`google-genai`	用 google.api_core.retry

Idempotency

`OpenAI: idempotency-key`	header,24h 内幂等
`Anthropic: idempotency-key`	同名,UUID 即可
`Gemini`	无原生,自行去重

退避公式

`delay = min(cap, base · 2^n) ± rand·jitter` ;base=1s,cap

高效技巧 — Pro Tips

- 用**官方 SDK**不用 raw HTTP——重试/流式/类型都是免费的
- 结构化输出用 **Pydantic**:OpenAI `response_format` ,Gemini `response_schema` ,Anthropic 走 tool_use
- **Anthropic Prompt Caching**: `cache_control` + 1h TTL,读取 **−90%**,写入 +25%
- **OpenAI 自动 Prompt Cache**:prompt >1k token 自动命中,缓存命中 **−50%**
- **Gemini Context Caching**:长系统/RAG 文档显式建 cache + TTL,长文本省一大笔
- **Batch API** 异步任务直接 **50% 折扣**(OpenAI/Anthropic 都有),24h 完成
- tier / rate limit 看 **dashboard**,RPM/TPM/RPD 三维度,涨 tier 看消费 + 时间
- 统一接口用 **LiteLLM** / **Portkey**,一份代码切百家模型 + 观测
- 大上下文别全塞 system,Anthropic **system 数组**分块更利于 cache
- 流式 + 工具:OpenAI 用 `tool_calls` 累积,Anthropic 用 `input_json_delta` 拼接
- 本地调试:OpenAI `OPENAI_LOG=debug` ,Anthropic `ANTHROPIC_LOG=debug`
- token 计算用 `tiktoken` / `anthropic.count_tokens` / `client.models.count_tokens`
- 多 key 轮询防 429,async 并发用 `AsyncOpenAI` / `AsyncAnthropic`
- 选模型:**Opus** 推理 / **Sonnet** 通用 / **Haiku** 快;**GPT-5** 通才;**Gemini 2.5** 长 context
- 生产加 **OpenTelemetry**:Langfuse / Helicone / Phoenix 观察 token + 延迟